Module 24: Programming in C++

Inheritance:

Part 4: Phone Hierarchy

Instructors: Abir Das and Jibesh Patra

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

{*abir*, *jibesh*}*@cse.iitkgp.ac.in*

Slides taken from NPTEL course on Programming in Modern C++

by **Prof. Partha Pratim Das**

Module Objectives

- Model a hierarchy of phones using inheritance

Module Outline

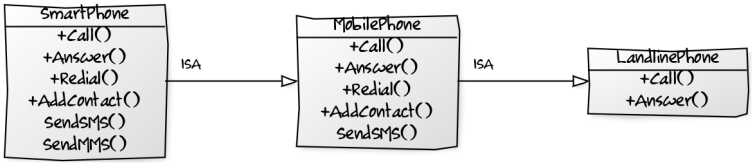
Approach to Modeling Hierarchy

- Identify the **Concepts and their ISA relationships** to define the hierarchy: model with public inheritance
- Identify and model **Helper classes** - lower level UDTs to define components
- Identify the **Interface** of each concept: signatures of public member functions
- Identify the **State Variables** of each concept: types of of private / protected data members (also member functions used for ease of implementation)
- **Refactor** common data members and member functions between specialized and generalized classes to link the classes by inheritance
- **Integrate the hierarchy** with abstract (pure) interface
- Explore **extendability**
- *We illustrate with the phone hierarchy*

Helper Classes

Class	Description
`class PhoneNumber`	12-digit phone number
`class Name`	Subscriber Name (as `string`)
`class Photo`	Image & Subscriber Name as alt text
`class RingTone`	Audio & ring tone name
`class Contact`	`PhoneNumber`, `Name`, and `Photo` (optional) of a contact
`class AddressBook`	List of contacts

Hierarchy of Phones



- `MobilePhone` **ISA** `LandlinePhone`
  - `LandlinePhone` is *generalization*
  - `MobilePhone` is *specialization*
  - `MobilePhone` inherits the properties of `LandlinePhone`
- `SmartPhone` **ISA** `MobilePhone`
  - `MobilePhone` is *generalization*
  - `SmartPhone` is *specialization*
  - `SmartPhone` inherits the properties of `MobilePhone`
- **ISA** is *transitive*

Interfaces of Phones

- **Landline Phone**
  - Call: By dial / keyboard
  - Answer
  - Caller ID (with special attached device)
- **Mobile Phone**
  - Call: By keyboard – shows number
    - By Number
    - By Name
  - Answer
  - Caller ID
  - Redial
  - Set Ring Tone
  - Add Contact
    - Number
    - Name
- **Smart Phone**
  - Call: By touchscreen – shows number & photo
    - By Number
    - By Name
  - Answer
  - Caller ID
  - Redial
  - Set Ring Tone
  - Add Contact
    - Number
    - Name
    - Photo

- There exists a substantial overlap between the functionalities of the phones
- A mobile phone is more capable than a land line phone and can perform (almost) all its functions
- A smart phone is more capable than a mobile phone and can perform (almost) all its functions
- **These phones belong to a Specialization / Generalization Hierarchy**

Interface & State Variable: Landline Phone

- **Landline Phone**
  - Call: By dial / keyboard
  - Answer

```
class LandlinePhone {
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;

public:
    LandlinePhone(const char *num, const char *subs);

    void Call(const PhoneNumber *p);

    void Answer();

    friend ostream& operator<<(ostream& os, const LandlinePhone& p);
};
```

Interface & State Variable: Mobile Phone

- **Mobile Phone**
  - Call: By keyboard – shows number
    - By Number
    - By Name
  - Answer
  - Redial
  - Set Ring Tone
  - Add Contact
    - Number
    - Name

```
class MobilePhone {
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();

public:
    MobilePhone(const char *num, const char *subs);

    void Call(PhoneNumber *p);
    void Call(const Name& n);

    void Answer();

    void ReDial();
    void SetRingTone(RingTone::RINGTONE r);
    void AddContact(const char *num = 0,
        const char *subs = 0);

    friend ostream& operator<<(ostream& os, const MobilePhone& p);
};
```

Interface & State Variable: Smartphone

- **Smart Phone**
  - Call: By touchscreen – shows number & photo
    - By Number
    - By Name
  - Answer
  - Redial
  - Set Ring Tone
  - Add Contact
    - Number
    - Name
    - Photo

```
class SmartPhone {
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();
    unsigned int size_;
    void DisplayPhoto();

public:
    SmartPhone(const char *num, const char *subs);

    void Call(PhoneNumber *p);
    void Call(const Name& n);

    void Answer();

    void ReDial();
    void SetRingTone(RingTone::RINGTONE r);
    void AddContact(const char *num = 0,
        const char *subs = 0);

    friend ostream& operator<<(ostream& os, const MobilePhone& p);
};
```

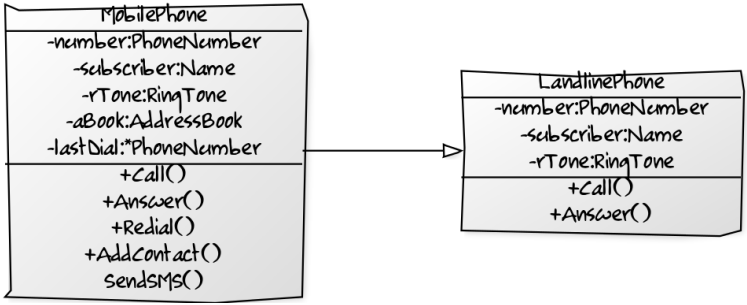
Refactoring

MobilePhone **ISA** LandlinePhone

```
class LandlinePhone { protected:
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;



public:
    LandlinePhone(const char *num,
        const char *subs) :
        number_(num), subscriber_(subs),
        rTone_() { }
    void Call(const PhoneNumber *p);

    void Answer();



    friend ostream& operator<<(ostream& os,
        const LandlinePhone& p);
};
```

```
class MobilePhone : public LandlinePhone { protected:
    //PhoneNumber number_;
    //Name subscriber_;
    //RingTone rTone_;
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();
public:
    MobilePhone(const char *num,
        const char *subs) :
        LandlinePhone(num, subs), // Base ctor
        lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Overload
    //void Answer();                 // Inherited
    void ReDial();
    void SetRingTone(RingTone::RINGTONE r);
    void AddContact(const char *num = 0,
        const char *subs = 0);
    friend ostream& operator<< (ostream& os,
        const MobilePhone& p);
};
```

MobilePhone **ISA** LandlinePhone



SmartPhone **ISA** MobilePhone


```
class MobilePhone : public LandlinePhone { protected:
    //PhoneNumber number_;
    //Name subscriber_;
    //RingTone rTone_;
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();


public:
    MobilePhone(const char *num,
        const char *subs) :
        LandlinePhone(num, subs), // Base ctor
        lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Overload
    //void Answer();                 // Inherited
    void ReDial();
    void SetRingTone(RingTone::RINGTONE r);
    void AddContact(const char *num = 0,
        const char *subs = 0);
    friend ostream& operator<< (ostream& os,
        const MobilePhone& p);
};
```

```
class SmartPhone : public MobilePhone { protected:
    //PhoneNumber number_;
    //Name subscriber_;
    //RingTone rTone_;
    //AddressBook aBook_;
    //PhoneNumber *lastDial_;
    //void SetLastDialed(const PhoneNumber& p);
    //void ShowNumber();
    unsigned int size_;
    void DisplayPhoto()
public:
    SmartPhone(const char *num,
        const char *subs) :
        MobilePhone(num, subs), // Base ctor
        lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Override
    //void Answer();
    void ReDial();                   // Override
    //void SetRingTone(RingTone::RINGTONE r);
    //void AddContact(const char *num = 0,
        //const char *subs = 0);
    friend ostream& operator<< (ostream& os,
        const SmartPhone& p);
};
```

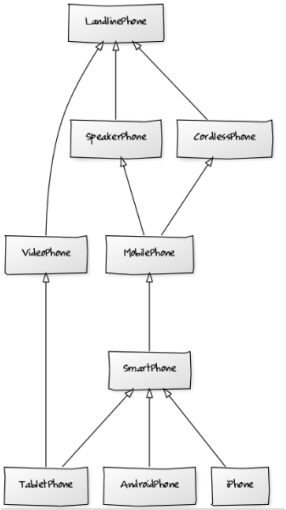
Hierarchy Integration

Hierarchy Integration


```
// Abstract Base Class - A Pure Interface
class Phone { public:
    virtual void Call(const PhoneNumber *p) = 0;
    virtual void Answer() = 0;
    virtual void ReDial() = 0;
};
class LandlinePhone: public Phone {
protected:
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;
public:
    LandlinePhone(const char *num,
        const char *subs) :
        number_(num), subscriber_(subs),
        rTone_() { }
    // Implementations for interfaces
    void Call(const PhoneNumber *p);
    void Answer();
    // Dummy implementation not for use
    void ReDial()
    { cout << "Not implemented" << endl; }
    friend ostream& operator<<(ostream& os,
        const LandlinePhone& p);
};
```

```
class MobilePhone : public LandlinePhone { protected:
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();
public:
    MobilePhone(const char *num, const char *subs) :
        LandlinePhone(num, subs), lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Overload
    void ReDial();                   // Override
    friend ostream& operator<< (ostream& os,
        const MobilePhone& p);
};
class SmartPhone : public MobilePhone {
protected: unsigned int size_;
    void DisplayPhoto();
public:
    SmartPhone(const char *num, const char *subs) :
        MobilePhone(num, subs), lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Override
    void ReDial();                   // Override
    friend ostream& operator<< (ostream& os,
        const SmartPhone& p);
};
```

Extended Hierarchy of Phones



Module Summary

- Using the Phone Hierarchy as an example analyzed the design process with inheritance

Module 25: Programming in C++

Inheritance: Part 5: `private` & `protected` Inheritance

Instructors: Abir Das and Jibesh Patra

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

{*abir, jibesh*}*@cse.iitkgp.ac.in*

Slides taken from NPTEL course on Programming in Modern C++

by **Prof. Partha Pratim Das**

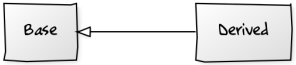
Module Objectives

- Explore restricted forms of inheritance (`private` and `protected`) in C++ and their semantic implications

Module Outline

Inheritance in C++: Semantics

- Derived **ISA** Base




```
class Base;                     // Base Class = Base
class Derived: public Base; // Derived Class = Derived
```

- Use keyword `public` after class name to denote inheritance
- Name of the Base class follow the keyword

Inheritance Exercise: What is the output?


```
class B {
public:
    B()  { cout << "B "; }
    ~B() { cout << "~B "; }
};

class C {
public:
    C()  { cout << "C "; }
    ~C() { cout << "~C "; }
};

class D : public B {
    C data_; // Embedded Object
public:
    D()  { cout << "D " << endl; } // Intrinsic Base Class Object
    ~D() { cout << "~D "; }
};

int main() {
    D d;
}
```

Inheritance Exercise: What is the output?

```
class B {
public:
    B()  { cout << "B "; }
    ~B() { cout << "~B "; }
};
class C {
public:
    C()  { cout << "C "; }
    ~C() { cout << "~C "; }
};
class D : public B {
    C data_; // Embedded Object
public:
    D()  { cout << "D " << endl; } // Intrinsic Base Class Object
    ~D() { cout << "~D "; }
};
int main() {
    D d;
}
```

Output:

```
B C D       // First base class object, then embedded object, finally self
~D ~C ~B
```

private Inheritance

- private Inheritance
  - Definition

    ```
    class Base;
    class Derived: private Base;
    ```

  - Use keyword private after class name
  - Name of the Base class follow the keyword
  - private inheritance does not mean generalization / specialization

private Inheritance

public Inheritance

```
class Person { ... };

class Student:
    public Person { ... };

void eat(const Person& p);     // anyone can eat
void study(const Student& s); // only students study

Person p;  // p is a Person
Student s; // s is a Student

eat(p);    // fine, p is a Person
eat(s);    // fine, s is a Student,
           // and a Student is-a Person

study(s);  // fine
study(p);  // error! p isn't a Student
```

- Compilers *converts* a derived class object (Student) into a base class object (Person) if the inheritance relationship is public

private Inheritance

```
class Person { ... };

class Student: // inheritance is now private
    private Person { ... };

void eat(const Person& p);     // anyone can eat
void study(const Student& s); // only students study

Person p;  // p is a Person
Student s; // s is a Student

eat(p);    // fine, p is a Person
eat(s);    // error! a Student isn't a Person
```

- Compilers *will not convert* a derived class object (Student) into a base class object (Person) if the inheritance relationship is private

Uncopyable Class

- Suppose we want to design a `MyClass` every object must be *unique*. That is instance objects must not be copied (the class needs to be Uncopyable)

```
MyClass m1;
MyClass m2;

MyClass m3(m1); // attempt to copy m1 - should not compile!
m1 = m2;        // attempt to copy m2 - should not compile!
```

- Naturally, we do not want to provide copy constructor or copy assignments operator. But that does not work as the compiler will provide free versions of these functions. How to stop that?

```
class MyClass {
public:
...
private:
...
    MyClass(const MyClass&); // declarations only
    MyClass& operator=(const MyClass&);
};
```

The last trick is not to provide the implementations (bodies) of these copy functions.

- With the above any global function or other class would not be able to copy - there will be *compilation error*; and any member function of `MyClass` will get *linker error* on missing implementation
- This is, of course, not an elegant solution and has to depend on the programmer to do things right for every such Uncopyable class. private inheritance helps out

Uncopyable

- `class Uncopyable` is designed as a root class that can make copy functions of any child class `private`

```
class Uncopyable { protected:  // allow construction and destruction of derived objects ...
        Uncopyable() { }
        ~Uncopyable() { }
    private:
        Uncopyable(const Uncopyable&); // ... but prevent copying
        Uncopyable& operator=(const Uncopyable&);
};
```

- Any class that inherits from `class Uncopyable`, will not have copy functionality:

```
class MyClass: private Uncopyable { // class no longer declares copy ctor or copy assign. operator
    // ...
    void ProhibitiveCopy() { MyClass test1, test2; // Member functions cannot perform copy
        MyClass test3(test1); // Error 1: 'Uncopyable::Uncopyable' : cannot access private member
        test2 = test1;        // Error 2: 'Uncopyable::operator =' : cannot access private member
    }
};
int main() { MyClass test1, test2; // Global functions cannot perform copy
    MyClass test3(test1); // Error 1: 'Uncopyable::Uncopyable' : cannot access private member
    test2 = test1;        // Error 2: 'Uncopyable::operator =' : cannot access private member
}
```

- **The inheritance from `Uncopyable` need not be `public` (though it will work), hence using `private` we express that it is purely for implementation - not for modeling ISA that actually does not exist**
- **C++11 provides an explicit support by `delete` to stop compilers from providing free functions**

Car HAS–A Engine: Composition OR `private` Inheritance?

Simple Composition

```
#include <iostream>
using namespace std;

class Engine {
public:
    Engine(int numCylinders) { }
    void start() { } // Starts this Engine
};
class Car {
public:
     // Initializes this Car with 8 cylinders
    Car() : e_(8) { }

    // Start this Car by starting its Engine
    void start() { e_.start(); }
private:
    Engine e_;  // Car has-a Engine
};
int main() {
    Car c;

    c.start();
}
```

`private` **Inheritance**

```
#include <iostream>
using namespace std;

class Engine {
public:
    Engine(int numCylinders) { }
    void start() { } // Starts this Engine
};
class Car : private Engine {  // Car has-a Engine
public:
     // Initializes this Car with 8 cylinders
    Car() : Engine(8) { }

    // Start this Car by starting its Engine
    using Engine::start;


};
int main() {
    Car c;

    c.start();
}
```

private Inheritance

- For **HAS-A**: Use composition when you can, private inheritance when you have to

> **• Private inheritance means nothing during software design, only during software implementation**
>
> **• Private inheritance means is-implemented-in-terms of. It is usually inferior to composition, but it makes sense when a derived class needs access to protected base class members or needs to redefine inherited virtual functions**
>
> – Scott Meyers in Item 32, Effective C++ (3rd. Edition)

protected Inheritance

protected Inheritance

- protected Inheritance
  - Definition

    ```
    class Base;
    class Derived: protected Base;
    ```

  - Use keyword protected after class name
  - Name of the Base class follow the keyword
  - protected inheritance does not mean generalization / specialization

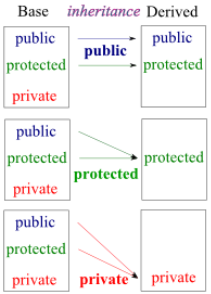
> • **Private inheritance means something entirely different (from public inheritance), and protected inheritance is something whose meaning eludes me to this day**
>
> – *Scott Meyers in Item 32, Effective C++ (3rd. Edition)*

Visibility

Visibility across Access and Inheritance

- Visibility Matrix

Visibility \ Inheritance	public	protected	private
public	public	protected	private
protected	protected	protected	private
private	-	-	-



Use and Examples

Inheritance Exercise: What is the output?


```
class B {
protected:
    B()  { cout << "B "; }
    ~B() { cout << "~B "; }
};
class C : public B {
protected:
    C()  { cout << "C "; }
    ~C() { cout << "~C "; }
};
class D : private C {
    C data_;
public:
    D()  { cout << "D " << endl; }
    ~D() { cout << "~D "; }
};

int main() {
    D d;
}
```

Inheritance Exercise: What is the output?


```
class B {
protected:
    B()  { cout << "B "; }
    ~B() { cout << "~B "; }
};
class C : public B {
protected:
    C()  { cout << "C "; }
    ~C() { cout << "~C "; }
};
class D : private C {
    C data_;
public:
    D()  { cout << "D " << endl; }
    ~D() { cout << "~D "; }
};

int main() {
    D d;
}
```

Output:

```
B C B C D
~D ~C ~B ~C ~B
```

Inheritance Exercise: Access Rights

Inaccessible Members

```
class A { private: int x;
    protected: int y;
    public: int z;
};
class B : public A { private: int u;
    protected: int v;
    public: int w; void f() { x; }
};
class C: protected A { private: int u;
    protected: int v;
    public: int w; void f() { x; }
};
class D: private A { private: int u;
    protected: int v;
    public: int w; void f() { x; }
};
class E : public B { public: void f() { x; u; }
};
class F : public C { public: void f() { x; u; }
};
class G : public D { public: void f() { x; y; z; u; }
};
```

Accessible Members

```
void f(A& a,
       B& b, C& c, D& d,
       E& e, F& f, G& g) {
    a.z;

    b.z;
    b.w;

    c.w;

    d.w;

    e.z;
    e.w;

    f.w;

    g.w;
}
```

Module Summary

- Introduced restricted forms of inheritance and `protected` specifier
- Discussed how `private` inheritance is used for *Implemented-As* Semantics

Module 26: Programming in C++

Polymorphism: Part 1: Type Casting

Instructors: Abir Das and Jibesh Patra

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

{*abir, jibesh*}*@cse.iitkgp.ac.in*

Slides taken from NPTEL course on Programming in Modern C++

by **Prof. Partha Pratim Das**

Module Objectives

- Understand type casting and the difference between implicit and explicit casting
- Understand type casting in a class hierarchy
- Understand the notions of upcast and downcast

Module Outline

Type Casting: Basic Notions

- Casting is performed when a *value (variable) of one type* is used in place of *some other type*. Converting an expression of a given type into another type is known as **type-casting**

```
int i = 3;
double d = 2.5;

double result = d / i; // i is cast to double and used
```

- Casting can be **implicit** or **explicit**

```
d = i;          // implicit: int to double
i = d;          // implicit: warning: '=' : conversion from 'double' to 'int': possible loss of data
d = (double)i; // explicit: int to double
i = (int)d;     // explicit: double to int
```

- Casting Rules can be grossly classified for:
  - Built-in types
  - Unrelated types
  - Inheritance hierarchy (static)
  - Inheritance hierarchy (dynamic)

Comparison of Implicit and Explicit Casting

Implicit Casting	Explicit Casting
• Done *automatically*	• Done *programatically*
• *No data loss*, for promotion Compiler will be *silent*	• Data loss *may or may not take place* Compiler will be *silent*
• *Possible data loss*, for demotion Compiler will issue *warning*	
• Requires *no special syntax*	• Requires *cast operator* for conversion *C style operator*: (< *type* >) *C++ style operators*: `const_cast`, `static_cast`, `dynamic_cast`, and `reinterpret_cast`
• *Avoid*, *if possible*	• *Avoid C style cast* *Use C++ style cast*
• *Possible only* in *static* time	• *Possible* in *static* as well as *dynamic* time
• *May be disallowed* for *User-Defined Types*, but *cannot be disallowed* for *built-in types*	• *May be defined* for *User-Defined Types*

Type Casting Rules: Built-in Type

- Various type castings are possible between built-in types

```
int i = 3;
double d = 2.5;

double result = d / i; // i is cast to double and used
```

- Casting rules are defined between numerical types, between numercial types and pointers, and between pointers to different numerical types and `void`
- Casting can be **implicit** or **explicit**

```
int i = 3;
double d = 2.5, *p = &d;

d = i;          // implicit: int to double
i = d;          // implicit: warning: '=' : conversion from 'double' to 'int': possible loss of data

d = (double)i; // explicit: int to double
i = (int)d;    // explicit: double to int

i = p;          // error: '=' : cannot convert from 'double *' to 'int'
i = (int)p;    // explicit: double * to int
```

Type Casting Rules: Built-in Type: Numerical Types

- Casting is *safe* for *promotion* (All the data types of the variables are upgraded to the data type of the variable with larger data type)

  `bool` → `char` → `short int` → `int` → `unsigned int` → `long` → `unsigned` → `long long` → `float` → `double` → `long double`

- Casting in built-in types *does not invoke* any conversion function. It only *re-interprets* the binary representation
- Casting is *unsafe* for *demotion* – may lead to loss of data

Type Casting Rules: Built-in Type: Pointer Types

- Implicit casting between different pointer types is not allowed
- Any pointer can be implicitly cast to `void*` (with loss of type); but `void*` cannot be implicitly cast to any pointer type
- Conversion between array and corresponding pointer is not type casting – these are two different syntactic forms for accessing the same data

```
int i = 1, *p = &i, a[10]; double d = 1.1, *q = &d; void *r;

q = p;          // error: cannot convert ‘int*’ to ‘double*’
p = q;          // error: cannot convert ‘double*’ to ‘int*’
q = (double*)p; // Okay
p = (int*)q;    // Okay

r = p;          // Okay to convert from ‘int*’ to ‘void*’
p = r;          // error: invalid conversion from ‘void*’ to ‘int*’
p = (int*)r;    // Okay

p = a;          // Okay by array pointer duality. p[i], a[i], *(p+i), *(a+i) are equivalent
a = p;          // error: incompatible types in assignment of ‘int*’ to ‘int[10]’
```

Type Casting Rules: Built-in Type: Pointer Types

- Implicit casting between pointer type and numerical type is *not allowed*
- However, explicit casting between pointer and integral type (`int` or `long` etc.) is a common practice to support various tasks like *serialization* (save a file) and *de-serialization* (open a file)
- Care should be taken with these explicit cast to ensure that the integral type is of the *same size* as of the pointer. That is: `sizeof(void*)` = `sizeof(`$<$ *integraltype* $>$`)`

```
int i, *p = 0; long j;

// sizeof(i) = sizeof(int) = 4
// sizeof(j) = sizeof(long) = 8
// sizeof(p) = sizeof(int*) = sizeof(void*) = 8

i = p;       // error: invalid conversion from ‘int*’ to ‘int’
p = i;       // error: invalid conversion from ‘int’ to ‘int*’

i = (int)p;  // error: cast from ‘int*’ to ‘int’ loses precision
p = (int*)i; // warning: cast to pointer from integer of different size

j = (long)p; // Okay
p = (int*)j; // Okay
```

- Here, the conversion should be done between int* and long and not between int* and int

Type Casting Rules: Unrelated Classes

- (**Implicit**) Casting between *unrelated classes is not permitted*

```
class A { int i; };
class B { double d; };

A a;
B b;

A *p = &a;
B *q = &b;

a = b;     // error: binary '=' : no operator which takes a right-hand operand of type 'B'
a = (A)b;  // error: 'type cast' : cannot convert from 'B' to 'A'

b = a;     // error: binary '=' : no operator which takes a right-hand operand of type 'A'
b = (B)a;  // error: 'type cast' : cannot convert from 'A' to 'B'

p = q;     // error: '=' : cannot convert from 'B *' to 'A *'
q = p;     // error: '=' : cannot convert from 'A *' to 'B *'

p = (A*)&b; // explicit on pointer: type cast is okay for the compiler
q = (B*)&a; // explicit on pointer: type cast is okay for the compiler
```

Type Casting Rules: Unrelated Classes

- **Forced** Casting between *unrelated classes is dangerous*

```
class A { public: int i; };
class B { public: double d; };

A a;
B b;

a.i = 5;
b.d = 7.2;

A *p = &a;
B *q = &b;

cout << p->i << endl; // prints 5
cout << q->d << endl; // prints 7.2

p = (A*)&b; // Forced casting on pointer: Dangerous
q = (B*)&a; // Forced casting on pointer: Dangerous

cout << p->i << endl; // prints -858993459:    GARBAGE
cout << q->d << endl; // prints -9.25596e+061: GARBAGE
```

Type Casting Rules: Inheritance Hierarchy

- Casting on a **hierarchy** is *permitted in a limited sense*

```
class A { };
class B : public A { };

A *pa = 0;
B *pb = 0;
void *pv = 0;

pa = pb; //  UPCAST: Okay

pb = pa; //  DOWNCAST: error: '=' : cannot convert from 'A *' to 'B *'

pv = pa; // Okay, but lose the type for A * to void *
pv = pb; // Okay, but lose the type for B * to void *

pa = pv; // error: '=' : cannot convert from 'void *' to 'A *'
pb = pv; // error: '=' : cannot convert from 'void *' to 'B *'
```

Type Casting Rules: Inheritance Hierarchy

- **Up-Casting** is *safe*

```
class A { public: int dataA_; };
class B : public A { public: int dataB_; };

A a;
B b;

a.dataA_ = 2;
b.dataA_ = 3;
b.dataB_ = 5;

A *pa = &a;
B *pb = &b;

cout << pa->dataA_ << endl;                      // prints 2
cout << pb->dataA_ << " " << pb->dataB_ << endl; // prints 3 5

pa = &b;

cout << pa->dataA_ << endl;                      // prints 3
cout << pa->dataB_ << endl;                      // error: 'dataB_' : is not a member of 'A'
```

Type Casting Rules: Inheritance Hierarchy

- **Down-Casting** is *risky*

```
class A { public: int dataA_; };
class B : public A { public: int dataB_; };

A a;
B b;

a.dataA_ = 2;
b.dataA_ = 3;
b.dataB_ = 5;

B *pb = (B*)&a;             // Forced downcast

cout << pb->dataA_ << endl; // prints 2
cout << pb->dataB_ << endl; // Compilation okay. Prints garbage for 'dataB_' -- no 'dataB_' in 'A' object
```

Module Summary

- Introduced type casting
- Understood the difference between implicit and explicit type casting
- Introduced the notions of Casting in a class hierarchy – upcast and downcast